LESSON 01 · CHEAT SHEET

Why Functional Programming in Java

Java 8 → Java 21 · shared dataset: 10 customers, 15 accounts, 60 transactions

Key rules

1. **Declarative = WHAT, imperative = HOW.** Streams state the result; the library owns the loop.
2. **Java is OO with functional features** (since Java 8), not a pure FP language: mutable by default, side effects allowed.
3. **Behaviour parameterisation:** write the loop once, pass the rule. Strategy class → anonymous class → lambda.
4. **Any interface with one abstract method** accepts a lambda, even one written before Java 8 (target typing).
5. **Pure function:** same input → same output, no side effects.
6. **Streams are lazy:** nothing runs until a terminal op (toList, count, reduce…). Elements flow one at a time.
7. **Money:** `BigDecimal`, scale 2, `RoundingMode.HALF_UP`. Never `double`.

Canonical snippets

1 · Imperative vs declarative Java 8

```
// HOW: loop + mutable total
BigDecimal total = BigDecimal.ZERO;
for (Transaction t : txns) {
    if (t.type() == TxnType.DEBIT) {
        total = total.add(t.amount());
    }
}
// WHAT: rule + result            -> 16692.44
txns.stream()
    .filter(t -> t.type() == TxnType.DEBIT)
    .map(Transaction::amount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
```

2 · Anonymous class → lambda (same old interface)

```
filter(txns, new TransactionFilter() { // 6 lines
    public boolean matches(Transaction t) {
        return t.amount()
                .compareTo(THRESHOLD) > 0;
    }
});
// lambda: 1 line, same result (11 txns)
filter(txns,
    t -> t.amount().compareTo(THRESHOLD) > 0);
```

Do / Don't

✔ Do

- Keep lambdas short; name rules
- Prefer method references when clearer
- Immutable data: records, `List.of`, `toList()`
- One stream op per line

✘ Don't

- Fill an outside list in `forEach`
- Put logic in `peek` (debug only)
- Expect output without a terminal op
- Turn every loop into a stream

API used in L01

API	Signature	Returns	Since
`Predicate<T>`	`boolean test(T t)`	boolean	8
`Function<T,R>`	`R apply(T t)`	R	8
`andThen`	`andThen(Function<? super R,? extends V>)`	Function<T,V>	8
`Stream.filter`	`filter(Predicate<? super T>)`	Stream<T>	8
`Stream.map`	`map(Function<? super T,? extends R>)`	Stream<R>	8
`Stream.reduce`	`reduce(T id, BinaryOperator<T>)`	T	8
`Collectors.toList`	`toList()`	List (no promise)	8
`List.of`	`of(E... e)`	immutable List	9
`Predicate.not`	`not(Predicate<? super T>)`	Predicate<T>	11
`Stream.toList`	`toList()`	unmodifiable List	16
`record`	`record Transaction(...)`	data carrier	16

3 · Modern Java 21 style

```
List<String> ids = txns.stream()
    .filter(HighValueRule::isHighValue)
    .map(Transaction::id)      // record [16]
    .toList();                 // unmodifiable [16]
long rest = txns.stream()      // -> 49
    .filter(Predicate.not(     // [11]
        HighValueRule::isHighValue))
    .count();
```

```
11 over 1000.00 [T003, T019, T025, T026, T027,
T038, T039, T042, T049, T055, T056]
```

4 · Functions as values Java 8

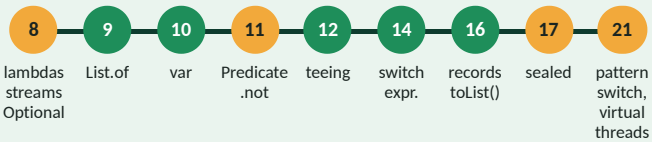
```
Function<BigDecimal, BigDecimal> ADD_GST =
    a -> a.multiply(new BigDecimal("1.09"))
          .setScale(2, RoundingMode.HALF_UP);
discountOf(10).andThen(ADD_GST)
    .apply(new BigDecimal("59.90"));   // 58.76
```

Laziness: the conveyor belt



Order printed: `filter T001, filter T002, filter T003, map T003, filter T004, map T004`. No terminal op → nothing prints. `limit(2)` found 2 high-value txns after 19 of 60.

Java 8 → 21 timeline



Amber = LTS. Beyond Java 21: unnamed variables _ (22), Stream Gatherers (24).

Lesson 01 · sticky notes

Cut out, stick on your wall, and read one every morning.

WHAT, not HOW

Declarative code states the result.
The library runs the loop for you.

1 abstract method = lambda

Any SAM interface takes a lambda, even an old one like `TransactionFilter`.

13 → 6 → 1

strategy class → anonymous class → lambda.
Same 11 transactions every time.

Lazy streams!

No terminal op = nothing runs.
`filter T003` → `map T003` before T004.

Pure function

Same input → same output.
No side effects. Easy to test.
`fee(2500.00) = 12.50` always.

toList() [Java 16]

Returns an unmodifiable list.
`add()` → UnsupportedOperationException

Money = BigDecimal

scale 2 · HALF_UP · never double.
`59.90 → 58.76` (−10% then GST)

Java is OO + FP

Functional features since Java 8.
Not pure FP: you choose the style.
LTS: 8 · 11 · 17 · 21